

Teste de chamadas simultâneas — SIPp + Asterisk (com áudio RTP)

Pacote para gerar **chamadas SIP simultâneas** a partir de um **ramal registrado**, contra um **Asterisk**, com **áudio real via RTP** (arquivo PCAP G.711 a-law) e duração aleatória por chamada.

Suporta regime constante: mantém N chamadas ativas indefinidamente, repondo cada uma que termina.

Validado com **SIPp v3.7.7-PCAP**.

Arquivos

Arquivo	Função
<code>run.sh</code>	Script único de configuração e execução. É aqui que você edita tudo.
<code>register.xml</code>	Cenário SIPp que registra o ramal (auth digest, 401). Rodado 1x.
<code>call.xml.template</code>	Template do cenário de chamada. Edite este, não o <code>call.xml</code> .
<code>call.xml</code>	Gerado automaticamente pelo <code>run.sh</code> a partir do template. Sobrescrito a cada execução.
<code>credenciais.csv</code>	Gerado pelo <code>run.sh</code> . Mantido por compatibilidade (<code>-inf</code>), mas as credenciais reais vão por <code>-au/-ap</code> .
<code>pcap/g711a.pcap</code>	Áudio G.711 a-law tocado durante as chamadas.

Pré-requisitos

SIPp compilado com suporte a PCAP play. Verifique:

```
sipp -v | head -3      # deve indicar PCAP
```

Se o comando não for encontrado, este pacote acompanha um binário do SIPp pronto. Basta instalá-lo no PATH:

```
sudo mv sipp /usr/local/bin/
sudo chmod +x /usr/local/bin/sipp
sipp -v | head -3      # confirme que agora responde
```

O `run.sh` procura o SIPp no PATH (command `-v sipp`) e aborta se não encontrar — ele não usa o binário do diretório atual.

Arquivo `pcap/g711a.pcap`. O `run.sh` tenta localizá-lo automaticamente em `/usr/share/sipp/pcap/` e similares.

Como usar

Edite o topo do `run.sh` e rode:

```
bash run.sh
```

Não é necessário `chmod +x` se você invocar com `bash`.

Configuração (bloco 1 do `run.sh`)

```
ASTERISK_IP="192.168.68.205"    # alvo de REDE (IP ou FQDN)
ASTERISK_PORT="5060"
TRANSPORT="u1"                  # u1=UDP  t1=TCP

SIP_DOMAIN="192.168.68.205"     # dominio da IDENTIDADE (From/To/Request-URI)

RAMAL="1002"
SENHA="..."
DESTINO="22221864"

LOCAL_IP=""                     # vazio = autodetecta
```

ASTERISK_IP vs SIP_DOMAIN — por que são separados

ASTERISK_IP é **para onde os pacotes vão** (alvo de rede). SIP_DOMAIN é **o domínio da identidade SIP**, que aparece no Request-URI e nos headers `From/To`.

Eles são variáveis distintas porque o SIPp resolve o ASTERISK_IP para um endereço IP. Se os cenários usassem `[remote_ip]` nos headers, o domínio viraria o IP resolvido — e um registrar que faz match por domínio rejeitaria o REGISTER. Com a separação, você pode apontar para um IP direto (sem DNS) mantendo o domínio correto na identidade.

Se o seu ambiente casa o endpoint por IP, basta usar o mesmo valor nos dois.

LOCAL_IP — cuidado com NAT

Deixe **vazio** na maioria dos casos. O SIPp autodetecta o IP da interface.

Não coloque aqui o seu IP público: o `-i` do SIPp faz `bind` no endereço, e o kernel recusa um IP que não pertence a nenhuma interface local (erro 99, `Address not available`). Se precisar atravessar NAT, o caminho é port-forward no roteador, não este campo.

Parâmetros do teste (bloco 2 do `run.sh`)

```
SIMULTANEAS=100                # teto de chamadas ativas ao mesmo tempo (-l)
TOTAL=0                        # 0 = ilimitado | >0 = encerra nesse total (-m)
RATE=50                        # novas chamadas por periodo (-r)
RATE_PERIOD=1000               # periodo da taxa em ms (-rp)

DURACAO_MIN_MS=10000           # piso da duracao sorteada
DURACAO_MAX_MS=60000           # teto da duracao sorteada
PCAP_MS=7000                   # duracao do pcap (informativo)
```

Duração da chamada (aleatória por chamada)

Cada chamada sorteia sua própria duração, com **distribuição uniforme** entre `DURACAO_MIN_MS` e `DURACAO_MAX_MS`. O range é respeitado com precisão de milissegundos.

Para duração **fixa**, use o mesmo valor nos dois campos:

```
DURACAO_MIN_MS=30000
DURACAO_MAX_MS=30000    # todas as chamadas com 30s
```

O sorteio é feito pelo `<pause distribution="uniform" min max>` do próprio SIPp — ver [Duração aleatória: pause, não sample](#) para o motivo.

Áudio: toca uma vez, no início

O `pcap/g711a.pcap` (~7s) toca **uma vez** no começo da chamada. Depois disso o canal fica aberto sem RTP até o BYE.

Isso é suficiente para testar sinalização, ocupação de canal e capacidade. **Não serve para medir média** (jitter, perda, carga de RTPengine) em chamadas longas, porque só os primeiros 7s têm fluxo RTP.

O SIPp não tem loop nativo de PCAP. Manter RTP contínuo exigiria empilhar blocos de `play_pcap_audio` + `pause`, o que torna a duração múltipla de `PCAP_MS` (7s, 14s, 21s...) e reintroduz o problema de terminação descrito adiante. A escolha aqui foi **priorizar o range exato**.

`PCAP_MS` é apenas informativo (aparece no banner) e não afeta a duração. Se trocar o arquivo, meça a duração real com:

```
capinfos pcap/g711a.pcap | grep -i duration
```

Fluxo constante: manter N chamadas ativas

O `-l` do SIPp não é só um teto — é um **patamar que ele mantém preenchido**. Quando uma chamada termina, uma vaga abre e ele cria outra imediatamente. Se 17 chamadas encerram juntas, ele dispara 17 novas.

Para rodar em regime constante até você parar:

```
SIMULTANEAS=100    # patamar mantido
TOTAL=0            # sem -m: nao ha condicao de parada
RATE=50            # velocidade de reposicao
```

Com `TOTAL=0` o script omite o `-m` e o SIPp roda indefinidamente. Para encerrar:

• — saída suave: para de criar chamadas e aguarda as ativas terminarem

• **Ctrl+C** — imediato, deixa chamadas penduradas no Asterisk

RATE é o que estabiliza (ou não) o patamar

A taxa limita a **velocidade de reposição das vagas**. Se muitas chamadas terminarem ao mesmo tempo e o `RATE` for baixo, o patamar afunda e leva tempo para recompor:

RATE	Tempo para repor 17 vagas
5/s	~3,4s
50/s	~0,34s

Mantenha `RATE` folgado — como referência, pelo menos metade de `SIMULTANEAS` por segundo.

Se quiser uma **taxa específica** em vez de patamar (ex.: 5 chamadas a cada 10 segundos):

```
RATE=5
RATE_PERIOD=10000
SIMULTANEAS=50      # com folga: precisa acomodar o acumulo em regime
```

O acúmulo em regime é aproximadamente $(\text{RATE} / \text{RATE_PERIOD}) \times \text{duração_média}$. Se `SIMULTANEAS` for menor que isso, o teto trava a taxa antes dela ser alcançada.

Comece sempre com `SIMULTANEAS=1 / TOTAL=1` para validar que uma chamada completa antes de subir a carga.

Por que o `call.xml` é gerado a partir de um template

O `<pause>` é lido quando o SIPp **carrega** o cenário — antes de as variáveis `[$. . .]` existirem. Seus atributos exigem números literais.

Por isso o `run.sh` substitui `@@DURACAO_MIN_MS@@` e `@@DURACAO_MAX_MS@@` no template e gera o `call.xml` final a cada execução. Se sobrar algum placeholder, o script aborta em vez de rodar um cenário inconsistente.

Edite o `call.xml.template`, nunca o `call.xml` — este último é sobrescrito.

Detalhes de implementação que importam

Estes pontos custaram tempo para descobrir e não são óbvios na documentação do SIPp.

Transporte: `-t u1`, **não** `-u1`

O modo de transporte é a opção `-t` com o valor separado. `-u1` não existe e faz o SIPp imprimir a tela de ajuda.

Credenciais: `-au/-ap`, **não dentro do** `[authentication]`

O bloco `[authentication]` é substituído por um placeholder de tamanho fixo e preenchido por offset.

Keywords aninhadas nos seus argumentos corrompem o buffer — o resultado é um `username=.X` e conteúdo da linha anterior vazando para dentro do bloco.

Portanto: use `[authentication]` **sem argumentos**, com as credenciais vindo de `-au` (usuário) e `-ap` (senha) na linha de comando.

Só é permitido **um** `[authentication]` por cenário.

Variáveis: `-set`, **não** `-key`

bullet `-set nome valor` → referenciada como `[$nome]` (global variable)

bullet `-key nome valor` → referenciada como `[nome]` (generic parameter)

Os cenários aqui usam `[$. . .]`, portanto `-set`.

O SIPp valida a relação nos **dois sentidos** e aborta em qualquer descasamento:

bullet `-set` de variável não declarada → *"Can not set the global variable X, because it does not exist"*

bullet variável declarada no `<Global>` e não usada no corpo → *"Variable \$X is referenced 1 times!"*

Ou seja: o conjunto passado por `-set`, o declarado em `<Global variables="...">` e o usado no corpo do cenário precisam ser **idênticos** em cada cenário. Por isso o `register.xml` declara `domain`, `user` e o `call.xml`, `domain`, `user`, `dest`.

Duração aleatória: `pause`, **não** `sample`

A ação `<sample distribution="uniform">` produz um valor **double**, não inteiro. Usá-la para sortear um contador de repetições e depois comparar com `<test compare="less_than" variable2="...">` **não termina de forma confiável** — o loop passa do alvo e a chamada estoura o range configurado.

Sintoma: com `DURACAO_MIN_MS=15000 / DURACAO_MAX_MS=25000`, apareciam chamadas de mais de um minuto.

A forma correta de duração aleatória é a distribuição no próprio `<pause>`:

```
<pause distribution="uniform" min="15000" max="25000"/>
```

Aqui o SIPp trata o valor internamente como tempo, sem passar por comparação de variáveis. Outras distribuições disponíveis: `fixed`, `normal (mean/stdev)`, `exponential`, `lognormal`, `weibull`, `pareto`, `gamma`, `negbin`.

Salto condicional: `test`, **não** `condexec`

Se você reintroduzir um loop no cenário, use `test`:

```
<nop test="variavel" next="label"/>  <!-- salta se variavel for true -->
```

`condexec` **não** é salto condicional — ele apenas decide se o próprio nó executa. O `next` continua valendo como goto incondicional, resultando em loop infinito. Sintoma: a chamada nunca alcança o BYE e fica pendurada até o timeout.

`LOCAL_ARG[@]`: `unbound variable` em Bash antigo

Em Bash 4.3 e anteriores (RHEL/CentOS 7 e derivados), expandir um array **vazio** como `"${array[@]}"` sob `set -u` aborta com *"unbound variable"*. Bash 5 não reclama — daí o script funcionar numa VM e falhar em outra.

O `run.sh` usa a forma com fallback, que é segura nas duas versões:

```
${LOCAL_ARG[@]+"${LOCAL_ARG[@]}"}
```

Caractere `$` em ramal

O SIPp trata `$` de forma especial (*"Transaction names may not contain '\$' or ','"*). Um ramal como `108$1002` precisa ser escapado no `run.sh` (`RAMAL="108\$1002"`) e é o primeiro suspeito diante de comportamento estranho.

Cuidados no Asterisk

Limite de chamadas do endpoint — se o teste travar em 1-2 chamadas, verifique limites por AOR/endpoint no `pjsip.conf`:

```
[1002]
type=endpoint
device_state_busy_at=0          ; 0 = sem limite por device state
allow=!all,alaw,ulaw            ; o cenário oferece PCMA e PCMU
```

- bullet **qualify_frequency e o OPTIONS descartado** — o Asterisk faz keep-alive no contato registrado enviando OPTIONS. O SIPp não tem cenário para respondê-los e os descarta:

```
Discarding message which can't be mapped to a known SIPp call:
OPTIONS sip:1002@...
```

Isso é esperado e inofensivo em testes curtos. Em testes longos, o Asterisk marca o endpoint como *unreachable* e pode parar de encaminhar chamadas — desabilite o `qualify_frequency` no endpoint de teste.

- bullet **Destino que atende e sustenta áudio** — o DESTINO deve responder 200 OK e manter o canal aberto:

```
exten => 9999,1,Answer()
same => n,Echo()          ; devolve o audio (testa RTP nos dois sentidos)
same => n,Hangup()
```

- bullet **Capacidade de mídia** — 100 chamadas com áudio = 100 fluxos RTP. Verifique CPU, faixa de portas no `rtp.conf` e custo de transcodificação.

- bullet **Teste através de NAT não mede o Asterisk** — rodando de uma rede privada contra um IP público, você mede seu link e a tabela de conntrack do roteador. Para números confiáveis, o gerador precisa estar na mesma rede do Asterisk.

Interpretando os resultados

A tela do SIPp mostra o fluxo de mensagens por passo do cenário:

		Messages	Retrans	Timeout	Unexpected-Msg
0 :	REGISTER ----->	1	0	0	
2 :	401 <-----	1	0	0	0
3 :	REGISTER ----->	1	0	0	
4 :	200 <-----	E-RTD1 1	0	0	0

- bullet **Retransmissões altas com zero respostas** → o pacote não chegou ou está malformatado. Capture com `tcpdump` e inspecione o conteúdo.

- bullet **401 respondido mas o 2º REGISTER sem resposta** → o pacote autenticado está inválido. Verifique o header `Authorization` na captura.

- bullet `stats.csv` → métricas periódicas (chamadas/s, ativas, tempos de resposta)

- bullet `*_errors.log` → detalhe de cada falha

Teclas durante a execução: +/- muda a taxa, *// muda em 10x, q sai suave, p pausa.

Diagnóstico

Capture o tráfego real — é o que resolve a maioria dos casos:

```
# Em outro terminal, ANTES de rodar o run.sh
tcpdump -n -i enp0s3 host <ip_do_asterisk> -vv

# Ou com interface grafica de fluxo
sngrep -d any port 5060
```

Compare o que sai no pacote com o que está no XML. Divergência entre os dois (keyword não substituída, conteúdo vazando entre linhas) aponta problema de parsing do cenário, não de rede.

Um `[bad udp cksum]` no `tcpdump` é normal — é checksum offload da placa de rede, calculado depois da captura.

Ajustes rápidos

bullet **PBX desafia com 407 em vez de 401** → o 407 já está tratado como opcional nos cenários.

bullet **TCP** → mude `TRANSPORT` para `t1`.

bullet **Sem áudio, só sinalização** → remova o `<nop>` com `play_pcap_audio` no `call.xml.template` (mantenha o `<pause>`).

bullet **Duração fixa** → use o mesmo valor em `DURACAO_MIN_MS` e `DURACAO_MAX_MS`.

bullet **Verificar a duração real das chamadas** → o `CallLengthRepartition` no fim do relatório do SIPp mostra a distribuição; os buckets estão em `call.xml.template`.

Gerado para L5 Networks.